

Informe Proyecto Final

1st Julián Sánchez Ceballos
Facultad de ingeniería
Universidad de Antioquia
Medellín, Colombia
julian.sanchez6@udea.edu.co

2nd Jhon Alexander Tristancho Muñoz
Facultad de ingeniería
Universidad de Antioquia
Medellín, Colombia
jhon.tristancho@udea.edu.co,

Abstract—En este documento se presenta el proyecto final de la materia de informática II, el cual es un juego ambientado en tres acontecimientos históricos; La bomba en Nagasaki, el Bogotazo y el atentado del 11 de septiembre del 2001 (9/11).

En el juego se modelaron físicas, un agente inteligente y narrativas históricas de los acontecimientos. Se emplearon tres niveles distintos, cada nivel inspirado en uno de estos hechos históricos mencionados, estos niveles se desarrollaron con todos los conocimientos adquiridos en el curso; Programación orientada a objetos, interfaces gráficas de Qt, STL y memoria dinámica. Finalmente, se presenta toda la parte lógica empleada para el desarrollo de este juego y su implementación gráfica.

Index Terms—C++, programación orientada a objetos, videojuegos, STL, interfaces gráficas de Qt, memoria dinámica.

INTRODUCCIÓN

Como calificación final del curso de Informática II, se desarrolló un videojuego que nos permitió demostrar todas las habilidades de programación adquiridas en el curso. El objetivo principal fue crear un videojuego ambientado en hechos con un impacto significativo para la humanidad, que fuera atractivo para todos los usuarios en todos sus factores.

PRIMERA SOLUCIÓN PENSADA

Resumen del momento 1 y 2

Establecimos el juego en base a este momento, por eso es importante conocer que fue lo que se pensó:

- El programa fue pensado en busca de representar tres momentos históricos de gran impacto para la humanidad a través de experiencias interactivas breves, cada una con una vista y dinámica de juego diferente.
- Cada nivel cuenta con su propio sistema de físicas, objetivos concretos.

Requerimientos Funcionales

- 1) **Timer**, Se empleo el requisito del tiempo pedido, se modelo ese tiempo en el nivel uno y dos.
- 2) **Vistas**, En los tres niveles desarrollamos una vista claramente diferente en cada nivel, en el primer nivel se modelo una vista cenital fija, en el segundo una vista lateral fija y en el tercer nivel vista cenital tipo radar.
- 3) **Modelacion de físicas**
 - Movimiento oscilatorio vertical tipo seno.
 - Movimiento amortiguado.
 - Interpolación amortiguada hacia un objetivo con inercia.

- Movimiento parabólico con gravedad.

Clases y sus relaciones.

TABLE I
RESUMEN DE CLASES DEL JUEGO

Clase	Atributos	Métodos
GameController	QGraphicsView *m _{view}	showMenu(), showInstructions(int), startLevel(int), showEnding(), showGameOver(QString)
Level1Scene	punteros a Player, DogAgent, listas de fuegos, timers, textos de HUD.	updateScene(), updateTimer(), setView(), señales levelCompleted, levelFailed.
Level2Scene	Player *player, lista de AnimatedSprite* enemies, rocas, balas.	spawnEnemy(), updateScene(), updateTimer(), keyPressEvent(), keyReleaseEvent(), stopMusic(), endLevel().
Level3Scene	QList<Plane> planes, Plane *selected, listas de dangerZones y safeZones, safeLandings.	updateScene(), keyPressEvent(), mousePressEvent(), spawnPlane(), spawnDangerZones().
AnimatedSprite	QPixmap sheet, characterOffset, cols, rows, currentFrame, currentRow, frameWidth, frameHeight, timer.	setAnimationRow(), start(), stop(), updateFrame().
Player	QPointF velocity, bool moving.	ctor (recibe nivel), keyPressEvent(), keyReleaseEvent(), advance(), getVelocity().
DogAgent	target, velocity, QList<QPointF> memory, dangerNearby, lastDir, lastRow, lastTarget, barkSound.	update(float dt), perceive(), act(), learn().
Plane	QPointF m _{velocity}	setState(), setVelocity(), state(), update(float dt), predictPos(float t).
Physics		oscillatory(), randomDrift(), dampedFollow(), projectile(), velocityToTarget().
MenuScene	m _{startCallback}	startMusic(), stopMusic(), mousePressEvent().
InstructionScene	titleText, instructionText.	ctor.

TABLE II
RELACIONES ENTRE CLASES

Clase origen	Relación	Clase destino
GameController	contiene	MenuScene
GameController	contiene	InstructionScene
GameController	contiene	Level1Scene
GameController	contiene	Level2Scene
GameController	contiene	Level3Scene
Level1Scene	contiene	Player
Level1Scene	contiene	DogAgent
Level2Scene	contiene	Player
Level3Scene	contiene	Plane
Player	Hereda	AnimatedSprite
DogAgent	Hereda	AnimatedSprite
Level1Scene	contiene	Physics
Level2Scene	contiene	Physics
Level3Scene	contiene	Physics

A. Arquitectura del programa

La arquitectura del juego se organizó en varias capas lógicas apoyadas en el framework de Qt y en el paradigma de programación orientada a objetos. El objetivo fue separar de forma clara la interfaz gráfica, la lógica de juego, las entidades del mundo y los módulos de apoyo físicas y de audio.

- **Presentación:** Está formada por la ventana principal (QGraphicsView) y las distintas escenas gráficas que se muestran al usuario: MenuScene, InstructionScene y las escenas de cada nivel (Level1Scene, Level2Scene, Level3Scene). Cada escena hereda de QGraphicsScene y se encarga de dibujar el fondo, los textos de HUD (tiempo, vidas, mensajes) y los objetos gráficos que participan en la interacción.
- **Capa de lógica de juego:** El componente central de esta capa es la clase GameController, responsable de coordinar el flujo general: muestra el menú, las instrucciones, inicia los niveles y gestiona las pantallas de finalización o derrota. Cada escena de nivel encapsula sus propias reglas: actualización del estado en cada fotograma, detección de colisiones, gestión del temporizador y decisión de victoria o derrota. La comunicación entre escenas y el controlador se realiza mediante señales y slots de Qt (por ejemplo, levelCompleted y levelFailed), lo que reduce el acoplamiento entre módulos.
- **Capa de entidades:** Aquí se agrupan las clases que representan elementos del mundo del juego y que se reutilizan en varios niveles: AnimatedSprite actúa como clase base para cualquier personaje animado con spritesheet, Player y DogAgent heredan de AnimatedSprite y añaden comportamiento específico del jugador y del agente autónomo. Plane representa a los aviones del nivel 3, con estados internos de vuelo y actualización de posición.
- **Capa de servicios y utilidades:** Incluye la clase estática Physics, que concentra los modelos de movimiento

(oscilatorio, aleatorio amortiguado, seguimiento amortiguado, parabólico), y los objetos compartidos de audio, QMediaPlayer y QMediaPlaylist.

- **Decisión de QList:** En el proyecto, QList resultó ser el contenedor más apropiado porque se ajusta directamente a la forma en que las escenas administran sus entidades en tiempo real. En clases como Level1Scene, Level2Scene y Level3Scene se manejan colecciones de objetos que cambian constantemente: rocas que se crean y destruyen, enemigos que entran y salen, aviones que desaparecen al llegar a zonas seguras, proyectiles que se eliminan tras una colisión, y zonas de peligro que se agregan y remueven mediante temporizadores. Esta lógica que pensamos, donde los elementos deben insertarse y eliminarse de manera frecuente dentro del mismo ciclo de actualización, QList nos permitió modificar la colección sin preocuparnos por invalidar punteros a QGraphicsItem, ya que los elementos almacenados mantienen direcciones estables incluso cuando la lista cambia de tamaño. Esto es especialmente importante porque gran parte del sistema trabaja con punteros (por ejemplo, Plane*, AnimatedSprite*, QGraphicsPixmapItem*) que se pasan entre funciones como updateScene(), predictConflicts(), spawnEnemy() o spawnPlanes(). Además, la mayoría de los recorridos que hace el juego sobre estas colecciones son secuenciales, unos ejemplos podrían ser verificar colisiones entre aviones, actualizar la posición de todas las rocas o iterar sobre cada enemigo, lo cual encaja naturalmente con la estructura de lista sin requerir acceso aleatorio por índice.

Estructura de directorios

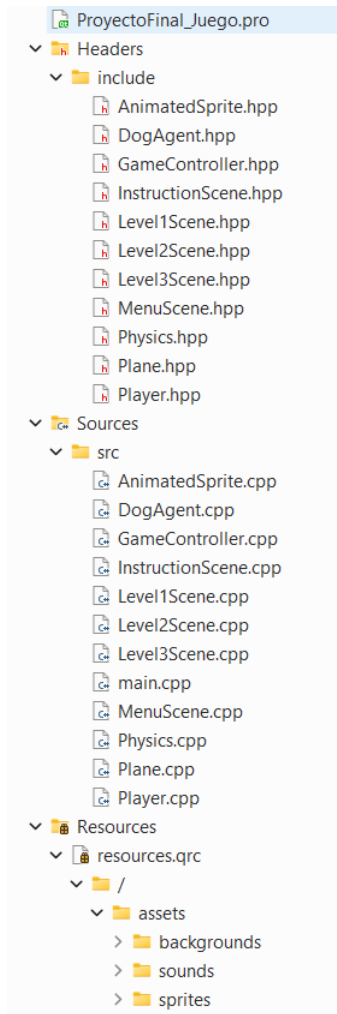


Fig. 1. Estructura final del proyecto

GESTIÓN Y FLUJO DEL JUEGO

El flujo principal comienza en `GameController`, que crea una instancia de `QGraphicsView` y la mantiene fija durante toda la ejecución. Dependiendo del estado del juego, el controlador sustituye la escena actual de la vista por:

MenuScene:

Es donde se le ofrece al usuario iniciar el juego y se reproduce música de fondo.

InstructionScene:

Esto es lo que muestra el contexto histórico y las instrucciones de control de cada nivel.

Niveles:

Cada escena de nivel dispone de uno o más `QTimer` que implementan el bucle de juego (actualización periódica de la escena) y el control de tiempo del nivel. Cuando se cumple una condición de victoria o derrota, la escena emite una señal

(`levelCompleted` o `levelFailed(QString)`), que es capturada por `GameController` para decidir si avanzar al siguiente nivel, mostrar un mensaje de fin de juego o volver al menú.

Este enfoque centrado en un controlador único y escenas independientes permite cambiar de nivel sin recrear la ventana principal ni mezclar la lógica de diferentes escenas.

Herencia desde AnimatedSprite:

`Player`, `DogAgent` y los enemigos del nivel 2 comparten la misma infraestructura de sprites animados: una textura dividida en filas y columnas y un `QTimer` interno que avanza los fotogramas. Cada subclase añade su propia lógica de entrada (teclas o decisiones del agente) y su reacción al entorno (colisiones, cambios de estado).

Uso de modelos de físicas parametrizables:

La clase `Physics` ofrece funciones estáticas que, dadas posiciones iniciales, tiempos, amplitudes, frecuencias o vectores de velocidad, devuelven nuevas posiciones. De esta manera: En el nivel 1, la columna de humo y el fuego utilizan movimiento oscilatorio y aleatorio, y el perro sigue al jugador mediante un seguimiento amortiguado. En el nivel 2, las rocas se mueven con trayectorias con gravedad ajustable. En el nivel 3, los aviones actualizan su posición según estados de vuelo que combinan movimientos rectilíneos, órbitas circulares y descensos acelerados.

Agente autónomo:

El agente autónomo del proyecto es el perro guía del nivel 1 (`DogAgent`). Su arquitectura interna separa cuatro componentes: primero la percepción, recibe la posición del jugador, la presencia de fuegos y la localización del refugio. Luego el razonamiento, decide si debe seguir al jugador o permanecer cerca del refugio en modo alerta. Ahora la acción, actualiza su posición y animación, e introduce señales visuales (cambio de color) y sonoras (ladrillo) cuando detecta que esta cerca el refugio. Y para finalizar el aprendizaje, dispone de una estructura de memoria donde puede almacenar posiciones de donde esta el refugio.

Soporte multimedia y portabilidad:

El juego integra audio y recursos gráficos a través del sistema de recursos de Qt, lo que permite empaquetar imágenes y sonidos dentro del proyecto. Las clases de escena gestionan la música de forma centralizada usando miembros estáticos, evitando solapamientos entre niveles.

Implementación de imágenes:

los fondos se añaden a cada `QGraphicsScene` como un `QGraphicsPixmapItem` escalado al tamaño total de la escena y con una profundidad (`ZValue`) muy baja, de manera que queden al fondo de todas las capas gráficas. A nivel de diseño, en cada nivel se buscaron paletas de color y composiciones coherentes con el hecho histórico que representan, que represente cada hecho importante, es muy importante que al momento de jugar se sienta que de verdad esta ambientado en ese hecho histórico. Se trato de buscar algo mas creativo, hechos que



Fig. 2. Menú de entrada del juego.



Fig. 4. Nivel 2.

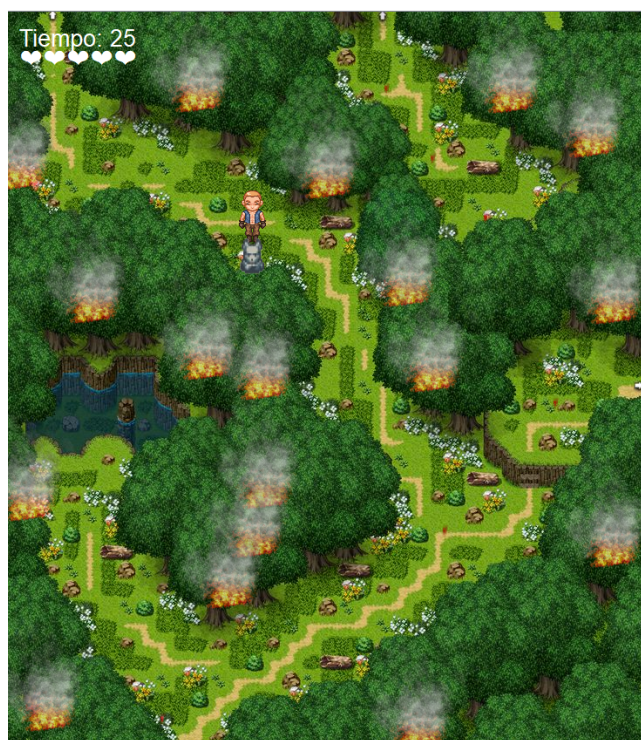


Fig. 3. Nivel 1.



Fig. 5. Nivel 3.

puedan resultar algo atractivos para los usuarios y eso se apoyo con el tráiler subido para apoyar la promoción del juego.

PROBLEMAS EN LA IMPLEMENTACIÓN

- **Compatibilidad con sistemas operativos:** En este caso el principal inconveniente que tuvimos fue trabajar en sistemas operativos distintos, al momento de desarrollar el juego como la versión de linux estaba trabajando con un compilador mas antiguo de algunas versiones que ya se tienen para windows, entonces nos toco investigar la manera de solucionar esto y desarrollar las rutas para los dos sistemas operativos.
- **Diseño visual:** Esta parte fue principalmente difícil por lo complejidad de encontrar varios grupos de sprites que se adaptaran de manera coherente al panorama del juego, entonces para nosotros fue difícil ver la manera en como acomodar toda la parte visual que se viera agradable.
- **Uso de la música:** Esta dificultad va muy relacionada con la parte de la version del compilador, dado que para un compilador mas reciente de Qt, no nos sirvieron unas librerias que eran para controlar la musica, pero al momento de usar una version mas antigua y descargando la libreria de multimedia pudimos dar solución con esto.

CONCLUSIONES

El desarrollo del juego permitió consolidar el uso de programación orientada a objetos en C++ dentro de un entorno gráfico realista como Qt, integrando físicas, animaciones y audio en un mismo proyecto. La implementación de un agente autónomo nos permitio demostrar ese componente investigador que tanto caracteriza a la universidad. Finalmente, en lo personal nos encanto que el tema a tratar fueran acontecimientos historicos, dado que asi nos pudimos inspirar y hacer un proyecto con hechos que nos llamaran la atención.

REFERENCES

- [1] Qt Company, *Qt Designer Manual (Qt 6)*, Qt Documentation, 2024. [Online]. Available: <https://doc.qt.io/qt-6/qtdesigner-manual.html>
- [2] Qt Company, *Graphics View Framework*, Qt Documentation, 2024. [Online]. Available: <https://doc.qt.io/qt-6/graphicsview.html>
- [3] Qt Company, *QGraphicsScene Class*, Qt Documentation, 2024. [Online]. Available: <https://doc.qt.io/qt-6/qgraphicsscene.html>
- [4] Qt Company, *QGraphicsItem Class*, Qt Documentation, 2024. [Online]. Available: <https://doc.qt.io/qt-6/qgraphicsitem.html>
- [5] Qt Company, *QGraphicsView Class*, Qt Documentation, 2024. [Online]. Available: <https://doc.qt.io/qt-6/qgraphicsview.html>
- [6] Qt Company, *Timers in Qt (QTimer)*, Qt Documentation, 2024. [Online]. Available: <https://doc.qt.io/qt-6/qtimer.html>
- [7] Qt Company, *Qt Multimedia Overview*, Qt Documentation, 2024. [Online]. Available: <https://doc.qt.io/qt-6/qtmultimedia-index.html>
- [8] Qt Company, *QMediaPlayer Class*, Qt Documentation, 2024. [Online]. Available: <https://doc.qt.io/qt-6/qmediaplayer.html>
- [9] Qt Company, *QMediaPlaylist Class*, Qt Documentation, 2024. [Online]. Available: <https://doc.qt.io/qt-6/qmediaplaylist.html>
- [10] Qt Company, *The Qt Resource System (qrc)*, Qt Documentation, 2024. [Online]. Available: <https://doc.qt.io/qt-6/resources.html>
- [11] Qt Company, *QPixmap Class*, Qt Documentation, 2024. [Online]. Available: <https://doc.qt.io/qt-6/qpixmap.html>
- [12] Qt Company, *QSoundEffect Class*, Qt Documentation, 2024. [Online]. Available: <https://doc.qt.io/qt-6/qsoundeffect.html>